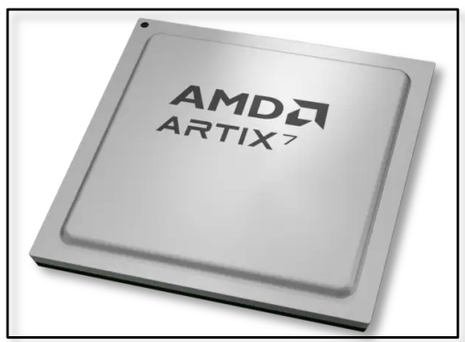
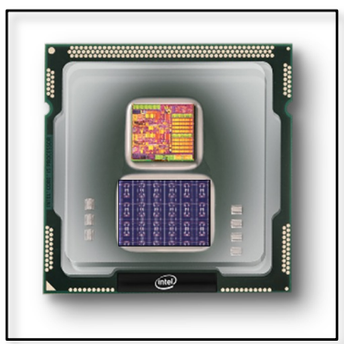
Christof Teuscher

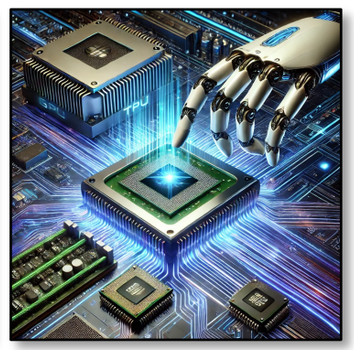
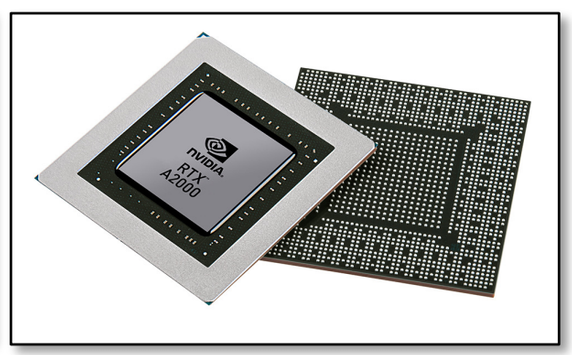
ECE 410/510: Hardware for AI and ML, Spring 2026

Course organization and details

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu





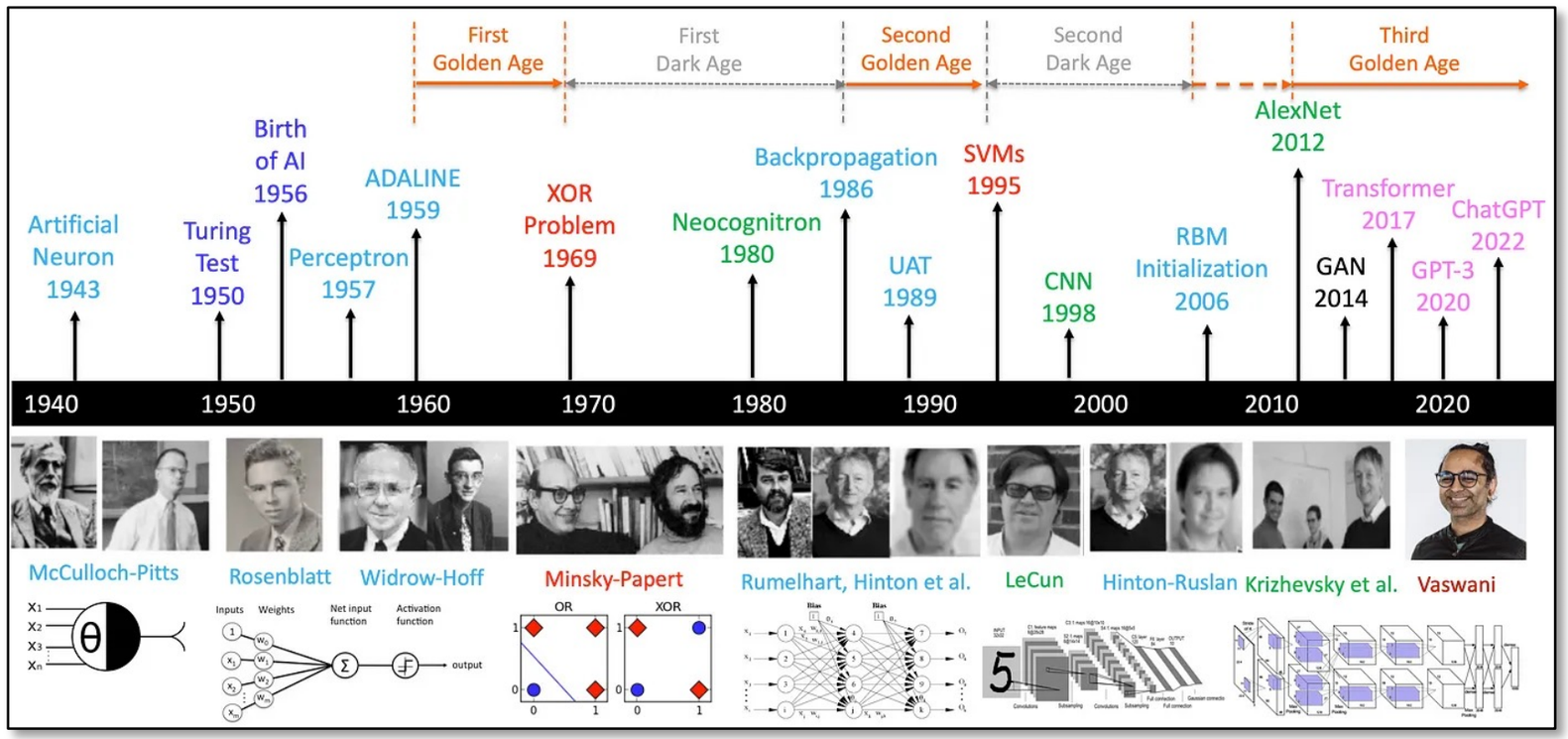




teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

Once upon a time…

Systems Science & ECE professor George Lendaris retired in 2011. With him, the one and only neural network course we offered died because there was not enough interest.



https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

ECE 486/586, Spring 2016

Final Project: A Simple Pipelined Neuromorphic Processor

Learning Goals

The main goal of this project is to expose you to the learning experience of designing a processor from the beginning to the end. You will explore different instruction set architectures for the task at hand, pick the most appropriate one, and then build a functional simulator of simple pipelined processor specialized to perform the operations for neural networks.

Additional learning goals:

- Explore design trade-offs.
- Learn how to build a functional simulator.
- Apply what we learned in class to a real-world problem.
- Propose a solution to a challenging problem you may not be familiar with.
- Expand your horizon, learn new things.
- Deal with ambiguity .
- Work on a team.
- Write a report.

Introduction and Motivation

Neuromorphic chips are chips that model functions similar to what the human brain performs. The following two articles provide an initial overview:

- http://semiengineering.com/neuromorphic-chip-biz-heats-up
- https://www.technologyreview.com/s/526506/neuromorphic-chips

There are many ongoing projects, both nationally and internationally, with the goal to replicate some of the brain's functionality at a large scale. Some projects are based on general-purpose processors (e.g., ARM processors for the SpiNNaker project, http://apt.cs.manchester.ac.uk/projects/SpiNNaker/project), while other projects chose to build highly specialized architectures based on novel types of devices (e.g., IBM's TrueNorth chip, http://www.research.ibm.com/articles/brain-chip.shtml, or Intel's spintronic-based approach, https://www.technologyreview.com/s/428235/intel-reveals-neuromorphic-chip-design).

For the purpose of this project, we will be working with a very simplified version of a neural network: feed-forward linear threshold neural networks. Note that most of today's neuromorphic architectures are non-linear and recurrent, but we do not need to go to that level of complexity to learn about the basic trade-offs a computer architect faces with neuromorphic architectures.

Your task is to design, model, simulate, test, and benchmark a simple pipelined neuromorphic processor for feed-forward linear threshold neural networks. The processor should be optimized for throughput, i.e., the more data you can feed through the neural network in a given time interval, the better.

To model, simulate, and test your processor design, you will write your own functional simulator in a high-level language of your choice (such as C, C++, Java, Python, etc.). The goal is not to build a simulator that is hardware-accurate for each gate or circuit involved. However, your simulator needs to be cycle-accurate, i.e., be able to capture accurately the total number of clock cycles a program takes to execute. If you prefer (in case you are not sufficiently familiar with a high-level language), you can use a hardware description language, such as VHDL or Verilog, and build a more realistic circuit-level simulation. Note that this is neither required nor will it get you any extra points.

teuscher.:Lab
teuscher-lab.com
Emerging Computing Models and Technologies
Christof Teuscher
ECE 486/586
Spring 2016
Portland State UNIVERSITY

All the typical parts of the architecture need to be simulated, such as the **instruction decoder**, **ALU**, **registers** (if applicable), **pipeline** (if applicable), data and instruction **memory**, etc. The functional simulator that you will build needs to capture the effect of running the simulated program on the simulated machine state (which includes the typical parts listed above).

Your simulator needs to be able to load a text file that contains a memory image as an input. The memory image represents the initial state of the simulated program. It needs to contain the program that simulates your neural network, the network weights, and the data to be fed into the network. That means you also have to simulate a basic memory (see more details below). The output of the simulator needs to be another memory image (dump) that shows output data of the neural network. The output can be appended to the input file or written into a separate file.

It is up to the team to define the file format that is appropriate for their implementation. The following is simply an example:

- The data in the memory image should be 32-bits wide. Each line in the text file represents one word (4 bytes) of memory shown in the hexadecimal (base-16) format. The addresses start at "0," from the first line in the image and then increase by "4" at every next line. The addresses do not need to be specified in the memory image and are implicitly obvious from the line number in the image file. For example, if you want to read the contents of memory address $(20)_{10}$, then those contents can be found on the sixth line in the memory image.

Assume that the PC and all the general purpose registers (if applicable) are initialized to "0." As you simulate each instruction in the program, you will need to keep track of the machine state and make necessary changes to the state based on the impact of the simulated instruction. For example, if an instruction writes a new value to register R6, then you will update the register R6 with the instruction result. Each instruction will also update the PC, which will in turn cause a new instruction to be fetched and simulated. You will continue the simulation until you encounter some sort of "HALT" instruction.

The Neural Network to be Simulated

Your task is to simulate a fixed feed-forward threshold neural network as depicted in Figure 1. The network has 4 input neurons, 4 hidden neurons, and 4 output neurons. Feed-forward means that the network does not have any cycles. In other words, it can be organized into layers that are only connected by "forward" connections as shown in Figure 1. There is no learning happening in this network. The weights are fixed and should be stored in the initial memory file.

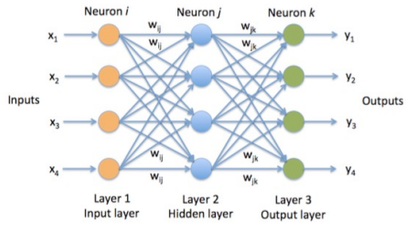


Figure 1. A 3-layer neural network with 4 input neurons, 4 hidden neurons, and 4 output neurons.

4/6/16 | rev 1

2016
ECE 586

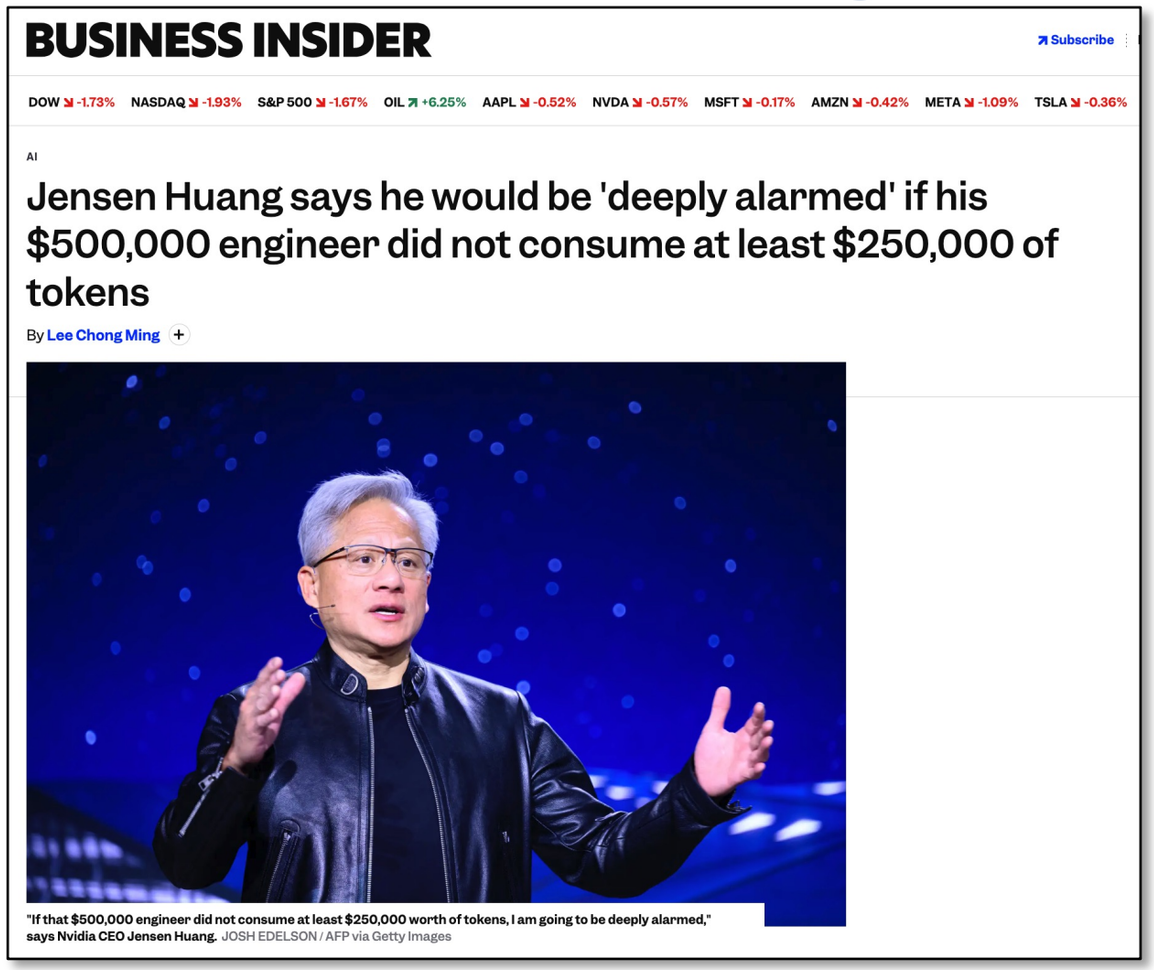
Why this course?

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

AI proficiency is not optional anymore

- AI is everywhere
- AI won't go anywhere
- On your job, you will be expected to use AI
- You need to be experts in getting work done with AI



BUSINESS INSIDER

Subscribe

DOW -1.73% NASDAQ -1.93% S&P 500 -1.67% OIL +6.25% AAPL -0.52% NVDA -0.57% MSFT -0.17% AMZN -0.42% META -1.09% TSLA -0.36%

AI

Jensen Huang says he would be 'deeply alarmed' if his $500,000 engineer did not consume at least $250,000 of tokens

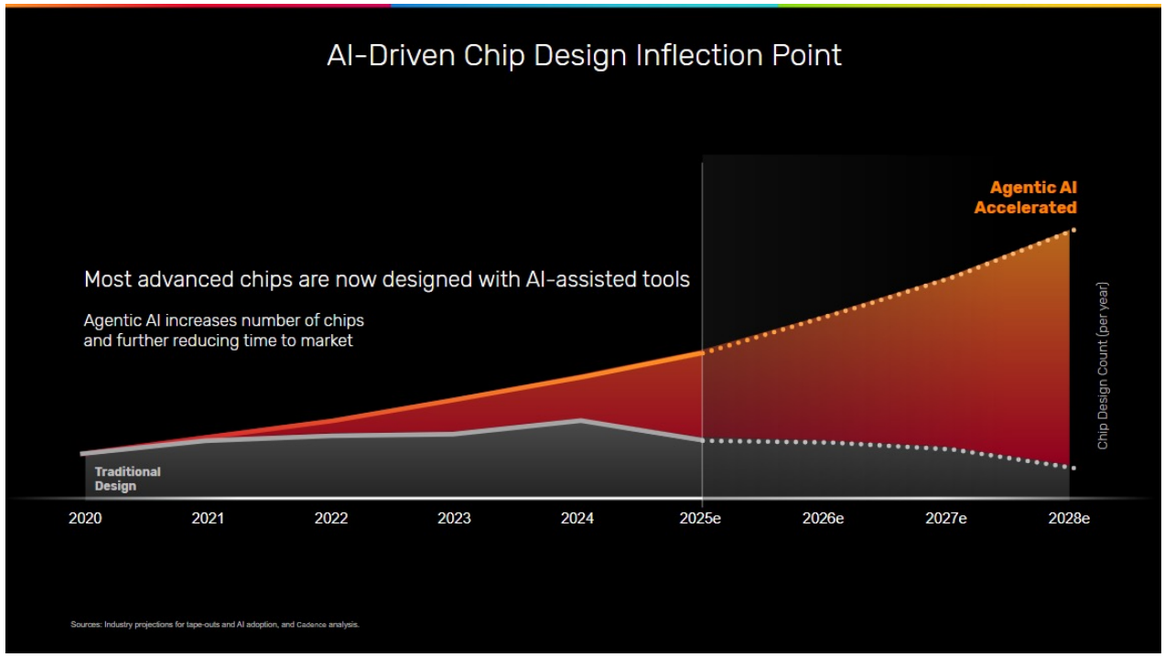
By Lee Chong Ming

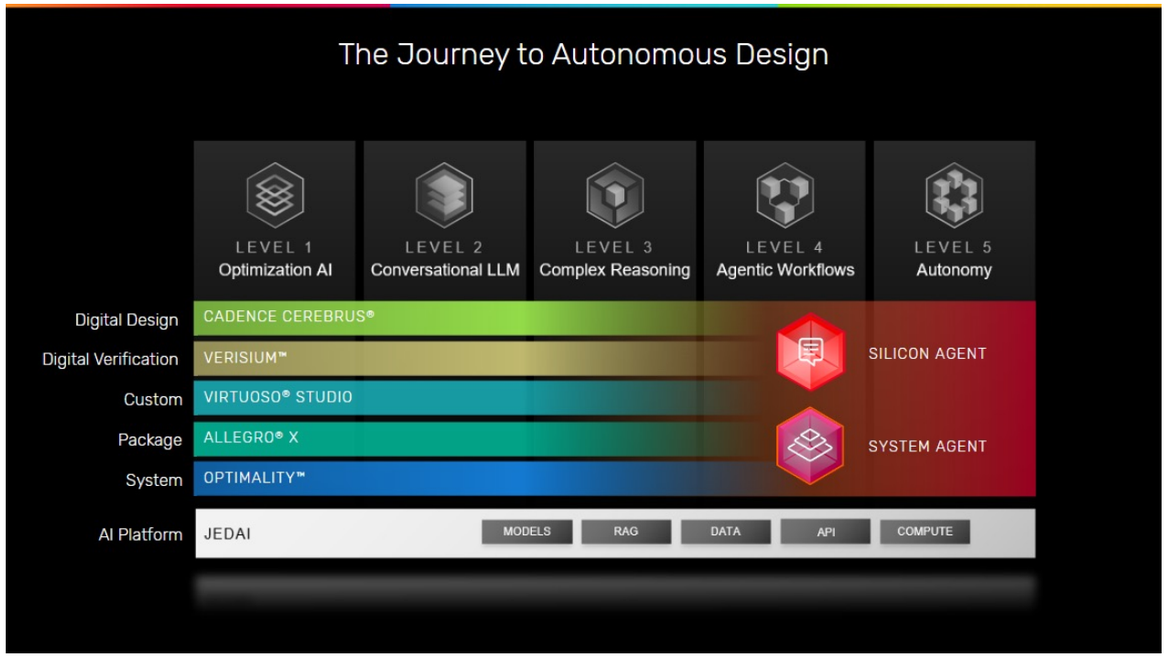
"If that $500,000 engineer did not consume at least $250,000 worth of tokens, I am going to be deeply alarmed," says Nvidia CEO Jensen Huang. JOSH EDELSON / AFP via Getty Images

https://www.businessinsider.com/jensen-huang-500k-engineers-250k-ai-tokens-nvidia-compute-2026-3

teuscher Lab
teuscher-lab.com

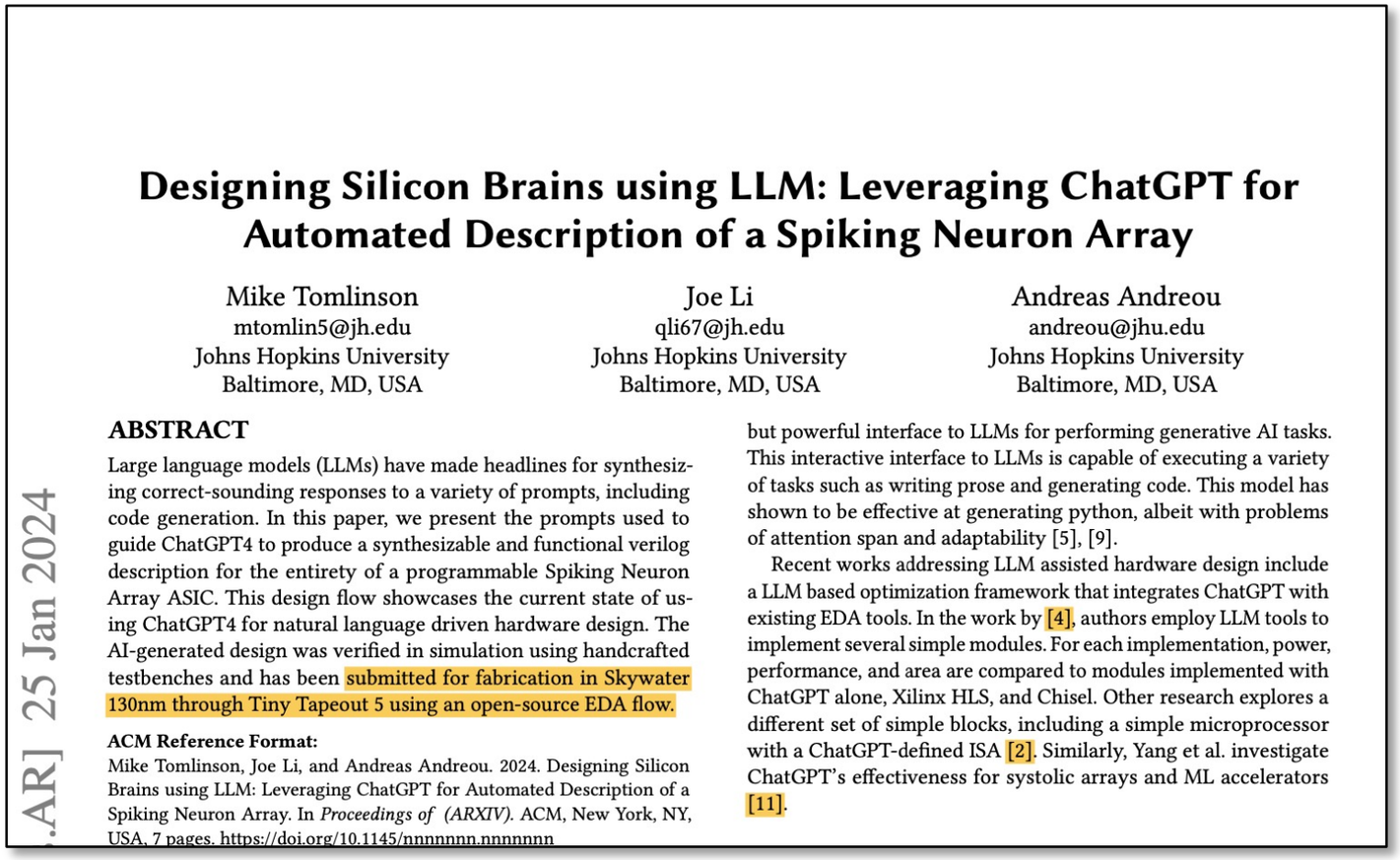
Portland State UNIVERSITY





https://community.cadence.com/cadence_blogs_8/b/corporate-news/posts/agentic-ai-soc-system-engineering

Inspiration and course design



Designing Silicon Brains using LLM: Leveraging ChatGPT for Automated Description of a Spiking Neuron Array

Mike Tomlinson
mtomlin5@jh.edu
Johns Hopkins University
Baltimore, MD, USA

Joe Li
qli67@jh.edu
Johns Hopkins University
Baltimore, MD, USA

Andreas Andreou
andreou@jhu.edu
Johns Hopkins University
Baltimore, MD, USA

ABSTRACT

Large language models (LLMs) have made headlines for synthesizing correct-sounding responses to a variety of prompts, including code generation. In this paper, we present the prompts used to guide ChatGPT4 to produce a synthesizable and functional verilog description for the entirety of a programmable Spiking Neuron Array ASIC. This design flow showcases the current state of using ChatGPT4 for natural language driven hardware design. The AI-generated design was verified in simulation using handcrafted testbenches and has been submitted for fabrication in Skywater 130nm through Tiny Tapeout 5 using an open-source EDA flow.

ACM Reference Format:
Mike Tomlinson, Joe Li, and Andreas Andreou. 2024. Designing Silicon Brains using LLM: Leveraging ChatGPT for Automated Description of a Spiking Neuron Array. In *Proceedings of (ARXIV).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

but powerful interface to LLMs for performing generative AI tasks. This interactive interface to LLMs is capable of executing a variety of tasks such as writing prose and generating code. This model has shown to be effective at generating python, albeit with problems of attention span and adaptability [5], [9].

Recent works addressing LLM assisted hardware design include a LLM based optimization framework that integrates ChatGPT with existing EDA tools. In the work by [4], authors employ LLM tools to implement several simple modules. For each implementation, power, performance, and area are compared to modules implemented with ChatGPT alone, Xilinx HLS, and Chisel. Other research explores a different set of simple blocks, including a simple microprocessor with a ChatGPT-defined ISA [2]. Similarly, Yang et al. investigate ChatGPT's effectiveness for systolic arrays and ML accelerators [11].

https://arxiv.org/abs/2402.10920

How do we teach cutting-edge new technology?

Nothing seems linear and predictable

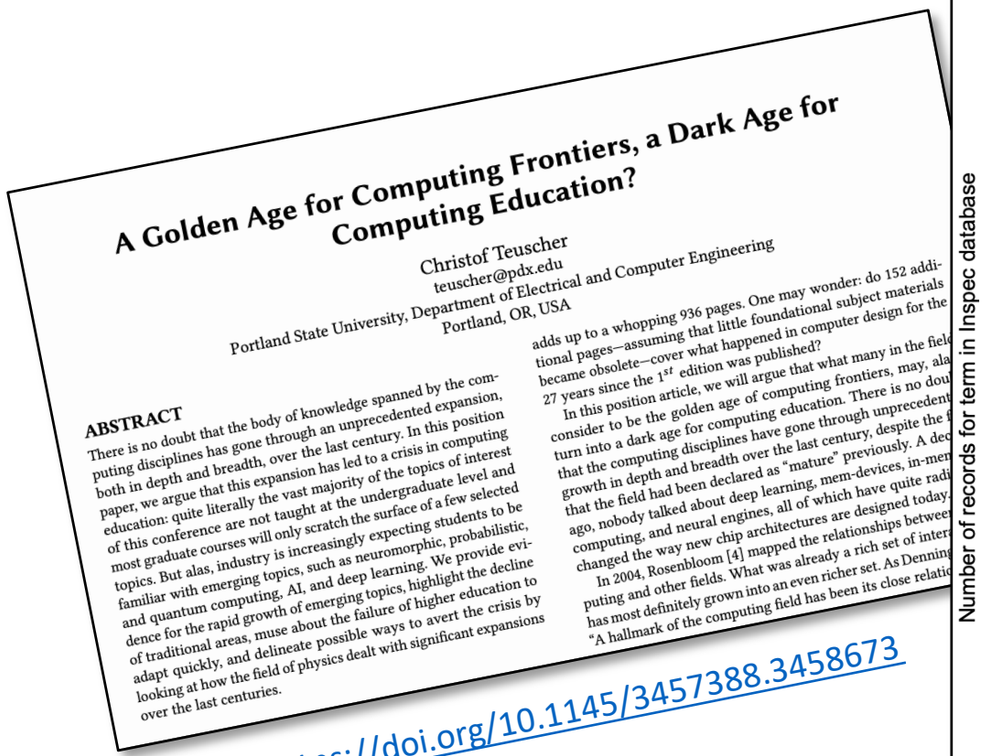
Everything is complex

Everything is connected

Everything moves fast

Christof Teuscher • teuscher@pdx.edu

The need for new educational approaches



A Golden Age for Computing Frontiers, a Dark Age for Computing Education?

Christof Teuscher
teuscher@pdx.edu
Portland State University, Department of Electrical and Computer Engineering
Portland, OR, USA

ABSTRACT

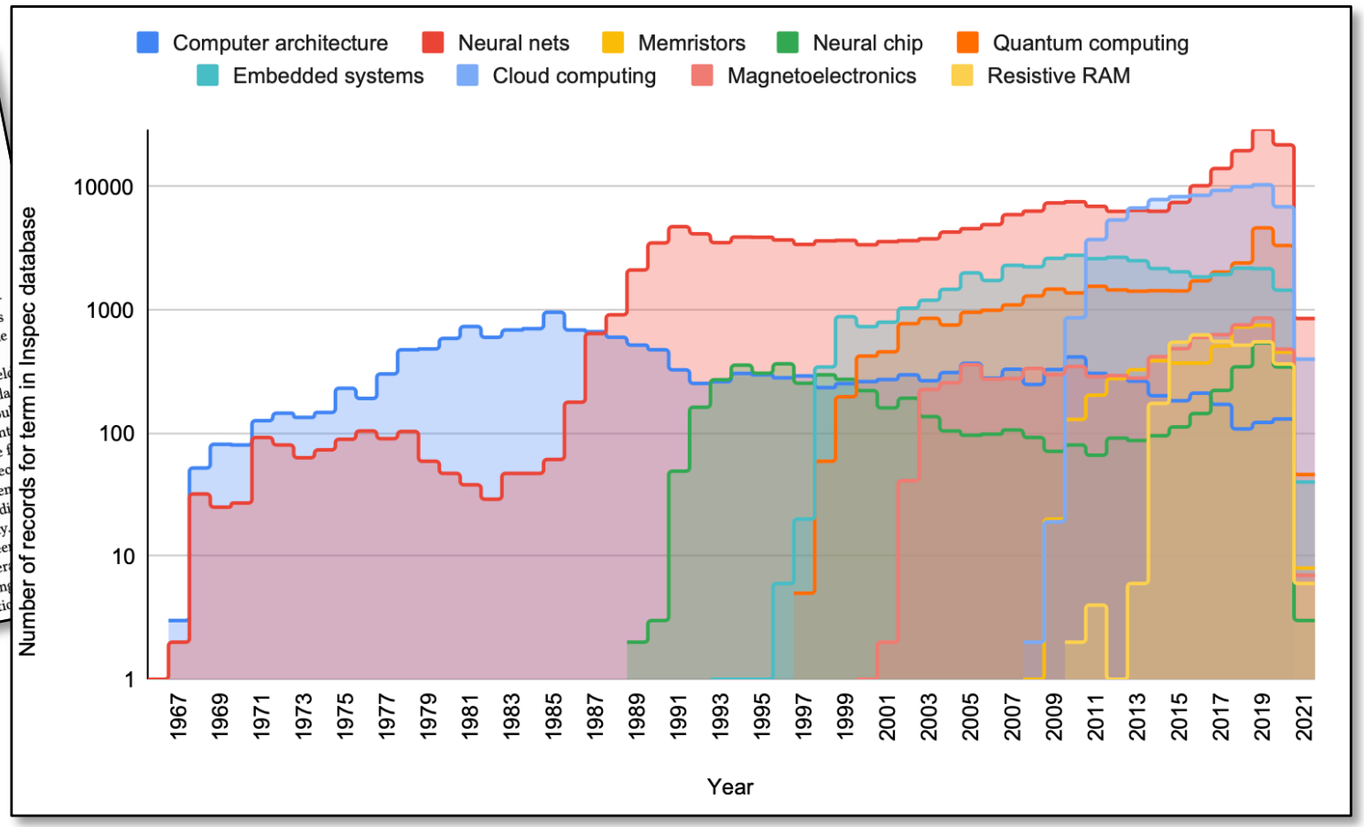
There is no doubt that the body of knowledge spanned by the computing disciplines has gone through an unprecedented expansion, both in depth and breadth, over the last century. In this position paper, we argue that this expansion has led to a crisis in computing education: quite literally the vast majority of the topics of interest of this conference are not taught at the undergraduate level and most graduate courses will only scratch the surface of a few selected topics. But alas, industry is increasingly expecting students to be familiar with emerging topics, such as neuromorphic, probabilistic, and quantum computing, AI, and deep learning. We provide evidence for the rapid growth of emerging topics, highlight the decline of traditional areas, muse about the failure of higher education to adapt quickly, and delineate possible ways to avert the crisis by looking at how the field of physics dealt with significant expansions over the last centuries.

adds up to a whopping 936 pages. One may wonder: do 152 additional pages—assuming that little foundational subject materials became obsolete—cover what happened in computer design for the 27 years since the 1st edition was published?

In this position article, we will argue that what many in the field consider to be the golden age of computing frontiers, may, ala turn into a dark age for computing education. There is no doubt that the computing disciplines have gone through unprecedented growth in depth and breadth over the last century, despite the fact that the field had been declared as "mature" previously. A decade ago, nobody talked about deep learning, mem-devices, in-memory computing, and neural engines, all of which have quite radically changed the way new chip architectures are designed today.

In 2004, Rosenbloom [4] mapped the relationships between computing and other fields. What was already a rich set of interactions has most definitely grown into an even richer set. As Denning stated, "A hallmark of the computing field has been its close relatio

https://doi.org/10.1145/3457388.3458673



teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

Table 2: List of graduate electives for an M.S. degree in computer engineering with a specialization in hardware and computer architecture at Boston University.

EC 513 Computer Architecture
EC 527 High-Performance Programming with Multicore and GPUs
EC 535 Introduction to Embedded Systems
EC 551 Advanced Digital Design with Verilog and FPGA
EC 561 Error-Control Codes
EC 571 VLSI Principles and Applications
EC 580 Modern Active Circuit Design
EC 582 RF/Analog IC Design Fundamentals
EC 713 Parallel Computer Architecture
EC 749 Interconnection Networks for Multicomputers
EC 752 Theory of Computer Hardware Testing
EC 753 Fault-Tolerant Computing
EC 757 Advanced Microprocessor Design
EC 772 VLSI Graduate Design Project

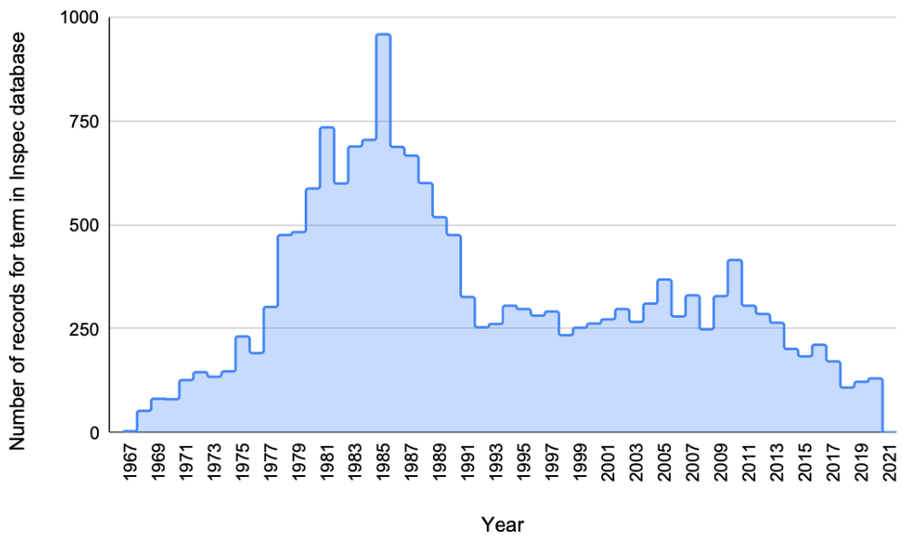


Figure 1: The number of records over the years that were found for the "computer architecture" thesaurus term. The field had its peak around 1985 and has since been in decline. Data source: Elsevier Engineering Village thesaurus.

Vibe Your Chip: An Unconventional Approach to Hardware Design Education in the Age of LLMs

Niklas Anderson[1*], Brandon Hippe[1,2], Tucker Mastin[1], Christof Teuscher[1]

[1*]Department of Electrical and Computer Engineering (ECE), Portland State University, 1900 SW 4th Ave, Portland, 97201, OR, US.
[2]Department of Electrical and Computer Engineering (ECE), Villanova University, 800 E. Lancaster Ave, Villanova, 19085, PA, US.

*Corresponding author(s). E-mail(s): niklas2@pdx.edu;
Contributing authors: bhippe@villanova.edu; tmastin@pdx.edu; teuscher@pdx.edu;

Abstract

The rapid proliferation of AI and machine learning applications has driven unprecedented innovation in computing hardware, from neuromorphic processors and memristive devices to quantum accelerators and in-memory computing architectures. However, traditional computer engineering curricula remain anchored in conventional digital design principles, creating a widening gap between industry demands and graduate competencies. This paper presents a pedagogical experiment for integrating emerging AI hardware paradigms into undergraduate and graduate computing education by relying heavily on Large Language Models (LLMs) for the design of AI accelerator hardware. Our assessments show that **89%** of surveyed students reported positive knowledge improvement ($M = 30.6$ points on a **100**-point scale), with consistent learning gains across graduate, undergraduate, and accelerated B+M students. Students produced substantial code artifacts, with a median of **2,847** lines of code per repository, and **90.5%** of repositories contained both Python and SystemVerilog or Verilog, reflecting genuine engagement with the hardware-software co-design pipeline. Students who entered with little or no hardware design experience produced working implementations of AI/ML accelerators and custom ASIC designs. Besides traditional lectures, the course design included "codefests," weekly presentations, self-grading, and unconventional incentives for students. All course materials

Last year's class

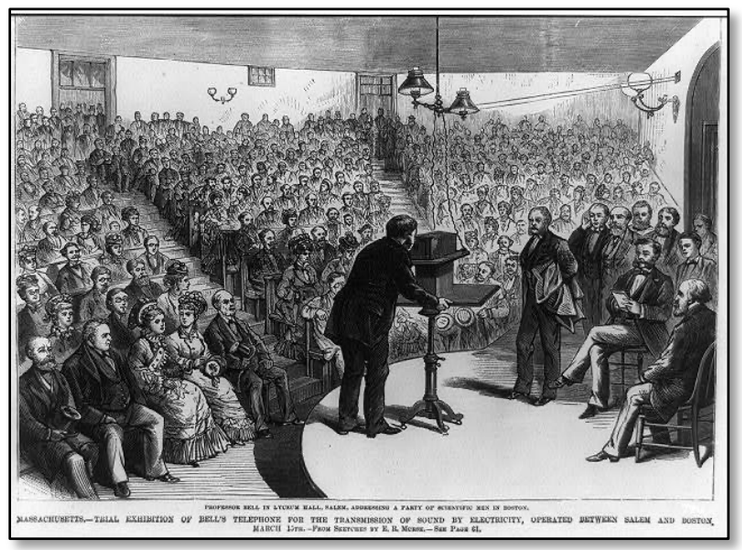
What's the role of education?

- **The old model**:
  - `knowledge + skills = success`
  - Domain-specific compartmentalization + professor with authority + lectures + fixed curriculum + assessment → **terminal degree**
- **The new model**:
  - creative thinking
  - complex problem solving
  - the ability to learn
  - emotional intelligence

    → **learning never ends**

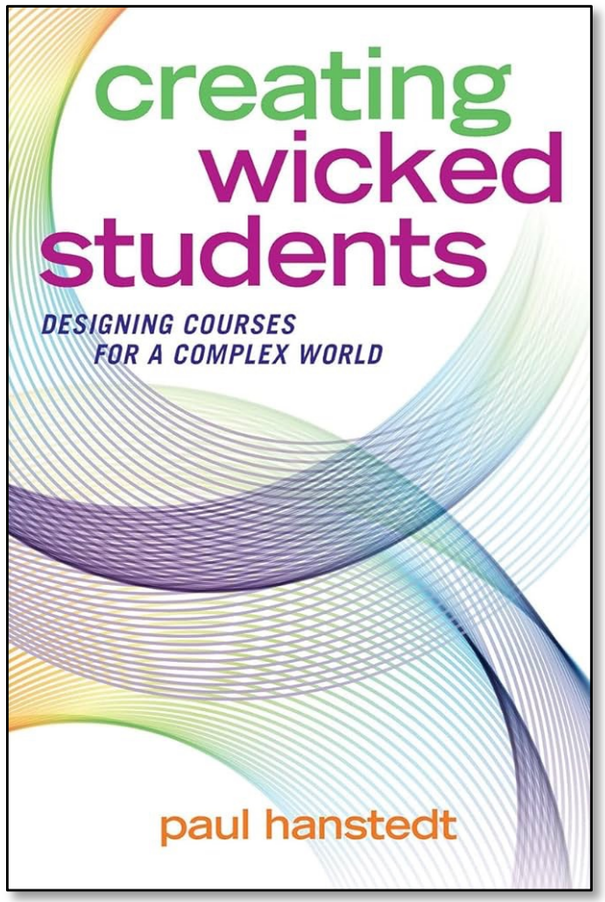
    → **be invested in your own learning**

    → **AI is your friend**



PROFESSOR BELL IN LYCEUM HALL, SALEM, ADDRESSING A PARTY OF SCIENTIFIC MEN IN BOSTON.

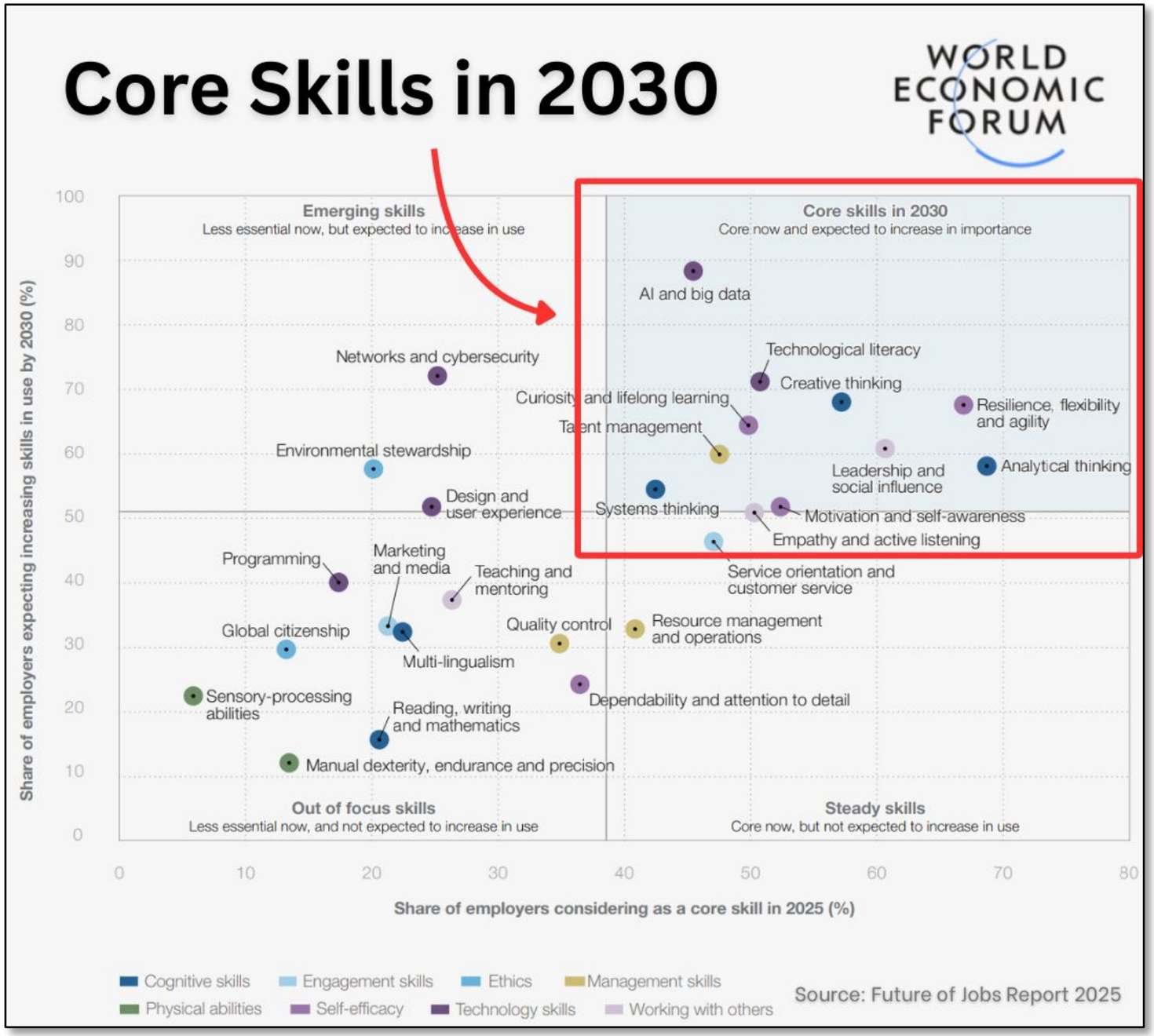
MASSACHUSETTS.—TRIAL EXHIBITION OF BELL'S TELEPHONE FOR THE TRANSMISSION OF SOUND BY ELECTRICITY, OPERATED BETWEEN SALEM AND BOSTON, MARCH 11TH.—FROM SKETCHES BY E. B. MORSE.—SEE PAGE 61.



A "wicked student," as coined by Paul Hanstedt, is a learner who takes authority over their own education, **capable of handling complex, ambiguous, and "wicked" problems, rather than just memorizing facts for a test**. They are **creative**, **adaptable**, and **comfortable with failure** as part of the learning process.

Core Skills in 2030

WORLD ECONOMIC FORUM



Source: Future of Jobs Report 2025

Let’s talk about the fine print…

My assumptions and expectations

- Intrinsically motivated to learn about this subject.
- Willing to be challenged, to fail on the way, and to get out of your comfort zone.
- Willing to show up and put in the work.
- Willing to not take shortcuts.
- Willing to always do a little more (rather than the bare minimum).
- Willing to assume responsibility for your actions (and/or non-actions).
- Willing to share and to engage.
- Willing to contribute to a positive learning environment

If you are here to

- …cheat
- …take shortcuts
- …take the easy route
- ...not be here

Well, you are the one who is missing out!

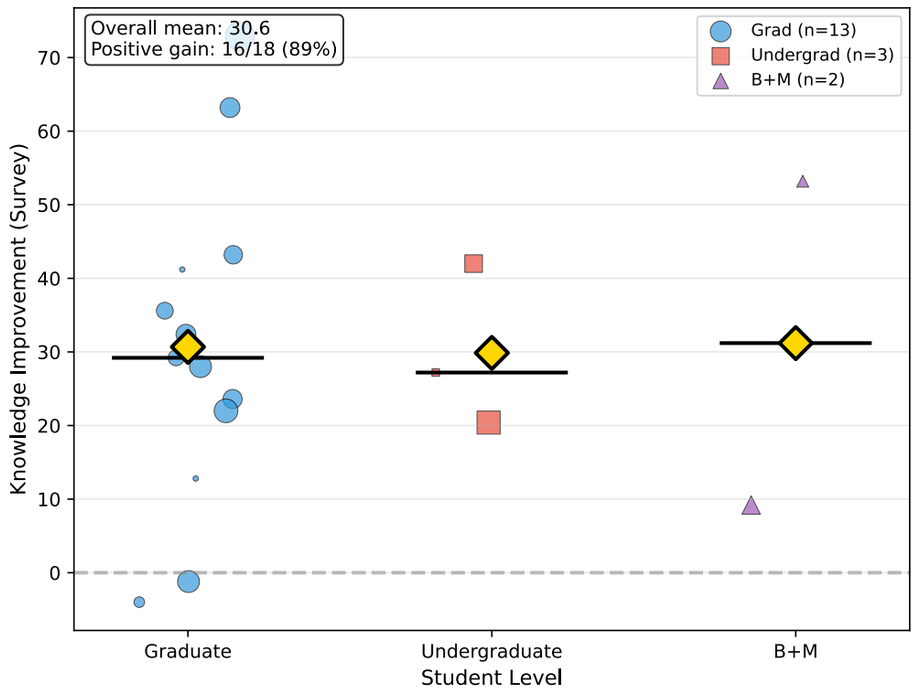
A word about prerequisites

Recommendations:

- Undergraduate: ECE 371 and ECE 351
- Graduate: ECE 485

Ideal knowledge:

- Python or other language
- Github
- Advanced computer architecture, incl. GPUs
- FPGAs
- Analog and digital circuits
- Verilog, SystemVerilog
- Experience with LLMs and vibe coding.



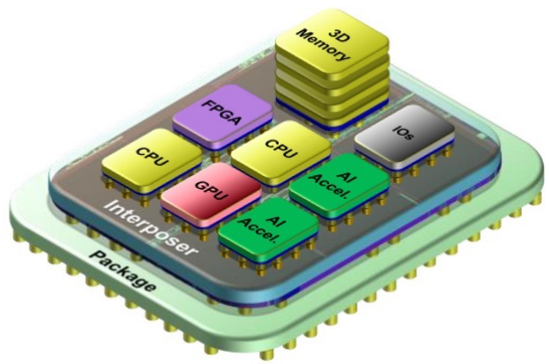
Learning goals

- Apply the mathematical foundations underlying AI/ML algorithms (linear algebra, calculus, probability) in the context of hardware design and optimization.
- **Explain the principles of HW/SW co-design and justify partitioning decisions for AI/ML workloads.**
- **Map AI/ML algorithms onto specialized hardware through co-design methodologies**.
- Evaluate and optimize HW designs through co-design for computational power, energy efficiency, and flexibility.
- Explain the foundations of neural networks and Large Language Models (LLMs), including CNNs, DNNs, and transformer architectures.
- Describe the architectures of specialized AI/ML hardware including GPUs, TPUs, FPGAs, neuromorphic processors, and in-memory computing systems.
- **Design, implement, and verify a custom hardware accelerator for an AI/ML algorithm using a hardware description language (Verilog or SystemVerilog).**
- **Use modern software and hardware tools, including LLM-assisted design flows, for designing and deploying specialized AI/ML hardware.**
- Critically evaluate LLM-generated hardware descriptions for correctness, efficiency, and synthesizability.
- Use the ASIC design flow (RTL to GDSII) including synthesis, placement, routing, and timing analysis.
- Benchmark hardware implementations against software baselines using appropriate performance metrics.
- Explain emerging computing paradigms, including in-memory computing, memristive devices, and their applications to AI/ML workloads.
- Communicate design decisions, tradeoffs, and results through professional documentation.

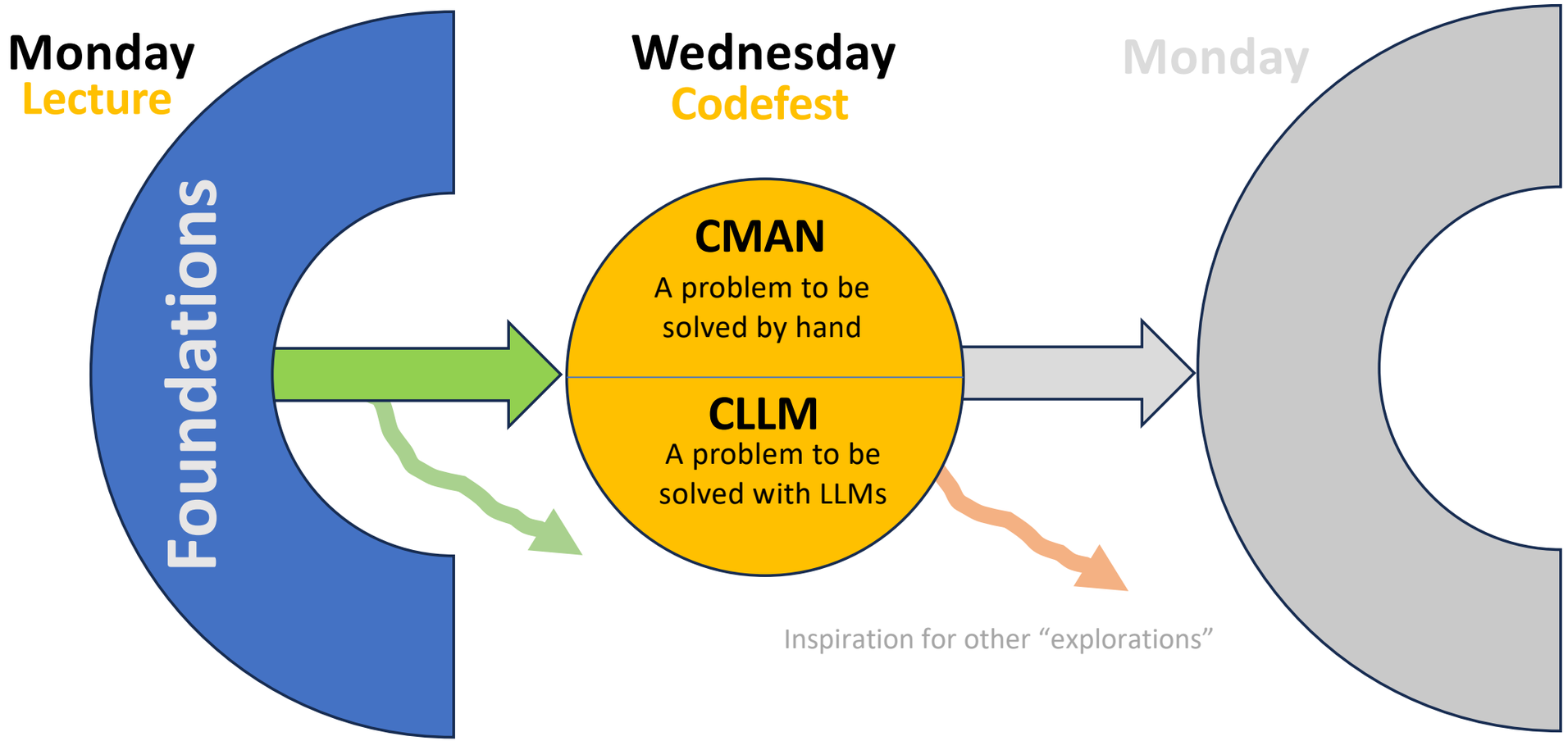
What this class is not

- Not a class about LLM, AI, ML, ANN, etc.
- …

You will each (co-)design, test, evaluate, etc. an AI/ML accelerator for a chiplet or for a standalone chip.



A typical week



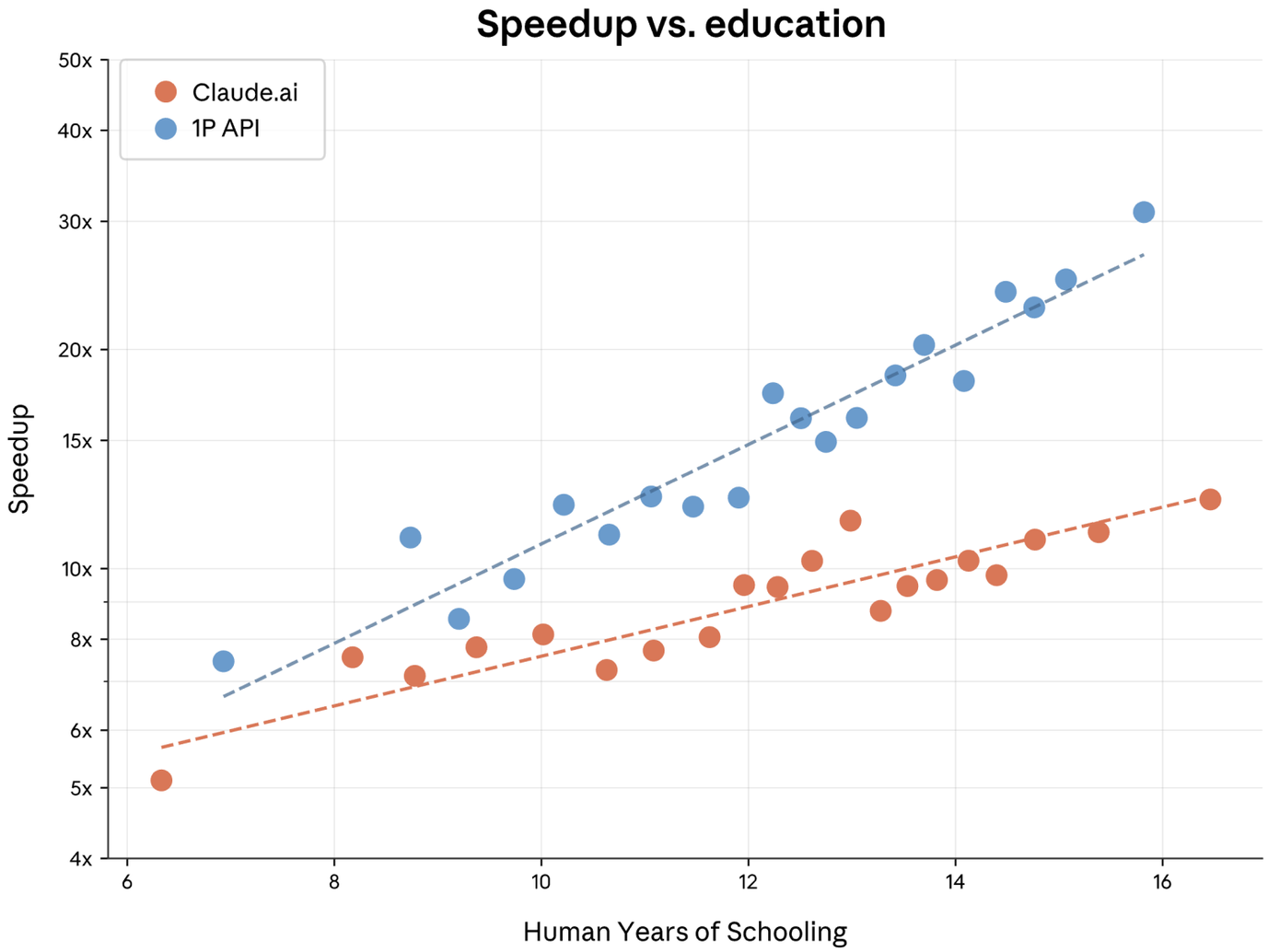
Pedagogical foundations (1)

Grounded in the literature.

The foundations are what lasts:

- Teach foundations, then multiply and scale them up them up with AI.

Christof Teuscher teuscher@pdx.edu



- The more you know, the more efficient you can use AI.
- AI is a **multiplier**, not a shortcut.
- **The foundations matter!**

How humans prompt is how Claude responds

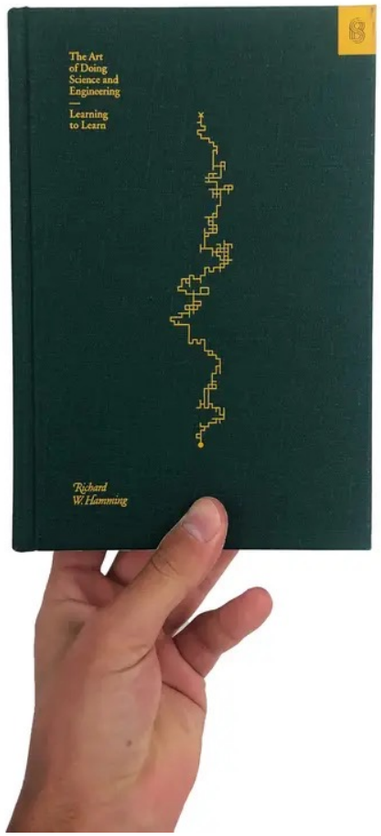
We find a very high correlation between human and AI education, i.e. the number of years of education required to understand a human prompt or the AI's response (countries: $r = 0.925$, $p < 0.001$, $N = 117$; US states: $r = 0.928$, $p < 0.001$, $N = 50$). This highlights the importance of skills and suggests that how humans prompt the AI determines how effective it can be. This also highlights the importance of model design and training. While Claude is able to respond in a highly sophisticated manner, it tends to do so only when users input sophisticated prompts.

https://www.anthropic.com/research/anthropic-economic-index-january-2026-report

Why learn if my LLM can do it for me?

- You can't just learn how to craft a prompt for an LLM without first having the experience, exposure and, yes, education to know what the heck you are doing.
- **The more sophisticated your prompt, the more sophisticated your answer**. Or in other words: Garbage in, garbage out.
- Learning is a messy, nonlinear human development process that resists efficiency. AI cannot replace it.

Does one need to understand AI-assisted designs?



"In science, if you know what you are doing, you should not be doing it.

In engineering, if you do not know what you are doing, you should not be doing it."

— Richard W. Hamming, **The Art of Doing Science and Engineering: Learning to Learn**.

Pedagogical foundations (2)

Problem-based or project-based learning

- Open-ended, real-world problems demand rigorous thinking (you can't fake a working solution) while allowing wildly different approaches.
- The constraint is the outcome quality, not the method.
- I'm not micromanaging how you get to the "solution."

Pedagogical foundations (3)

Structured autonomy:

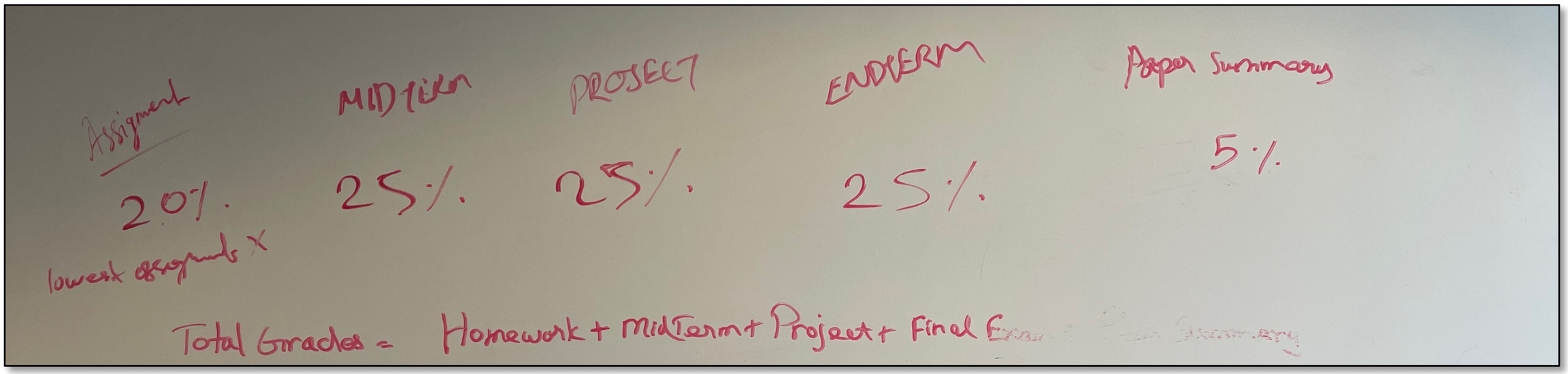
- Scaffold with clear learning objectives, non-negotiable competency thresholds, explicit criteria.
- You choose how to get there.
- Preserves accountability without micromanaging the path.
- Provide flexibility while keeping rigor.

Pedagogical foundations (4)

Low-stakes iteration, high-stakes demonstration.

- Keep practice low-pressure and revisable.
- Reserve high standards for culminating assessments.
- This gives students room to fail productively during learning while still holding a firm line at the end.

Traditional grading



This course uses specifications grading

- This course uses specifications grading.
- Each graded deliverable is evaluated as **Satisfactory (S)** or **Not Yet (NY)** against clearly defined specifications.
- Students who receive **Not Yet** can revise and resubmit.
- Final course grades are determined by which bundle of satisfactory work a student completes, not by accumulating points.

- Each student begins the term with **3 revision tokens**. A token is spent each time a student revises a **Not Yet** deliverable beyond the one free revision allowed per codefest deliverable and per milestone. Tokens cannot be used for Milestone 4 or the final examination.
- Additional tokens can be earned in the following ways:
  - Spot-check explanations: the instructor conducts 1–3 spot checks with every student during codefests across the term. A clear, correct verbal explanation over all spot checks earns 1 token.
  - Codefest presentations: every student presents their work at least once during the term. A strong presentation earns 1 token.
  - Consistent Slack contributions throughout the term, as assessed by the instructor, earns 1 token.

Codefest deliverables

- Each Wednesday codefest produces a **CMAN+CLLM deliverable pair** submitted to your GitHub repository by the following Sunday at 11:59 pm.

- **CMAN must be completed without AI assistance** and is submitted as a handwritten photo or plain-text file with all working shown.
- **CLLM scales the same concept to real-world complexity where AI** tooling is permitted and required.
- Each CMAN+CLLM pair is graded S/NY. Students receiving NY may revise and resubmit within one week of receiving feedback by using a revision token.

- Every student will present their work to the class at least once during the term; the instructor will schedule presentations across sessions.

Two Canvas quizzes

- **Week 5**: Quiz 1: Canvas-based, open-note, individual. Covers lecture material through Week 4.
- **Week 9**: Quiz 2: Canvas-based, open-note, individual. Covers lecture material through Week 8.
- Graded S/NY with an 80% threshold

Project milestones

- **The term-long co-design project is assessed at four milestones**. Each milestone is graded S/NY. Milestones 1 through 3 may be revised once with a revision token.

- **Milestone 1** (due Sun Apr 13): Refined Heilmeier answers, software profiling results, roofline analysis, HW/SW partition proposal, hardware interface selection with bandwidth justification, initial SW benchmark.
- **Milestone 2** (due Sun May 4): First working simulation with testbench; interface module in HDL with functional testbench; precision choice documented.
- **Milestone 3** (due Sun May 24): Synthesis attempt with timing and area report; interface integrated with compute core; co-simulation demonstrating end-to-end data flow; or justified scope adjustment.
- **Milestone 4** (due Sun Jun 7): Full deliverable package including source code, synthesis results, benchmark comparison, roofline plot, and design justification report. **This is final and cannot be revised.**

Final exam

- During week 10, every student submits their M4 package.
- An LLM examiner is provided the documentation and creates a set of exam questions on architectural decisions, tradeoffs, and applied concepts.
- Students are also assessed on their ability to identify when the AI examiner's questions are incorrect or misleading.

- The final exam format will be announced at a later stage.

- The final exam is graded **Pass with Distinction** / **Pass** / **No Pass**.
- **No retake is permitted.**

Undergraduate grade bundles (ECE 410)

Grade	Codefest deliverables (of 10)	Project milestones (of 4)	Final exam	Quizzes (of 2)
A	8+ S	All 4 S	Pass w/ Distinction	Both S
A−	7+ S	All 4 S	Pass	Both S
B+	6+ S	3+ S	Pass	Both S
B	5+ S	3+ S	Pass	1+ S
B−	4+ S	2+ S	Pass	1+ S
C+	3+ S	2+ S	Pass	0+ S
C	2+ S	1+ S	Pass	0+ S
C−	1+ S	1+ S	Pass	0+ S
D+	0 S	0+ S	Pass	0+ S
D	0 S	0+ S	Pass	0+ S
F	0 S	0 S	No pass	0 S

Graduate and B+M grade bundles (ECE 510)

Grade	Codefest deliverables (of 10)	Project milestones (of 4)	Final exam	Quizzes (of 2)
A	9+ S	All 4 S	Pass w/ Distinction	Both S
A−	8+ S	All 4 S	Pass	Both S
B+	7+ S	3+ S	Pass	Both S
B	6+ S	3+ S	Pass	1+ S
B−	5+ S	2+ S	Pass	1+ S
C+	4+ S	2+ S	Pass	0+ S
C	3+ S	1+ S	Pass	0+ S
C−	2+ S	1+ S	Pass	0+ S
D+	1+ S	0+ S	Pass	0+ S
D	1+ S	0+ S	Pass	0+ S
F	0 S	0 S	No pass	0 S

If M4 or the final examination is *No Pass,* the bundle grade drops by **2 letter grades** (e.g., A becomes B+). If both M4 and the final examination are No Pass, the grade drops by **4 letter grades** (e.g., A becomes B-). No revision is permitted for either deliverable.

"*A plumber doesn't get credit for effort; he gets credit if the faucet stops leaking*." – Seth Godin

Tentative course plan

Wk	Date	Day	Type	Topic / activity	Project / milestone
1	Mar 30	Mon	Lecture	Introduction: What is computation? What is a computer? Course organization. AI/ML landscape overview.	Identify candidate algorithms; review application list.
1	Apr 1	Wed	Codefest	CMAN (no AI): MAC/parameter/activation memory count by hand for a 3-layer MLP. CLLM (AI OK): Environment setup (GitHub, Slack). Profile ResNet-18 with torchinfo. Draft Heilmeier answers. Select project topic.	Draft Heilmeier answers; identify algorithm; run initial profiler; confirm SW baseline. Topic selected.
2	Apr 6	Mon	Lecture	HW/SW co-design fundamentals: partitioning, profiling bottlenecks, interface design, tradeoff analysis. Roofline model: arithmetic intensity, compute-bound vs. memory-bound regimes, ridge point.	Revisit HW/SW partition with roofline framing.
2	Apr 8	Wed	Codefest	CMAN (no AI): Roofline construction by hand; classify GEMM and vector-add kernels. CLLM (AI OK): CNN inference profiling; plot kernels on roofline; HW/SW partition proposal.	**M1 DUE Sun, Apr 12**: Heilmeier answers, profiling, roofline, HW/SW partition, interface selection, SW benchmark.
3	Apr 13	Mon	Lecture	GPU architecture: SIMT execution, warp scheduling, memory hierarchy, CUDA programming model. Memory wall: DRAM vs. SRAM, bandwidth constraints, energy cost of data movement.	Identify which operations in your algorithm map to SIMT or tensor-core patterns.
3	Apr 16	Wed	Codefest	CMAN (no AI): DRAM traffic analysis for naive vs. tiled 64×64 matrix multiply. CLLM (AI OK): CUDA GEMM naive vs. tiled; Nsight profiling; roofline placement.	Draft HW architecture due: block diagram, data path sketch, first HDL scaffold.

…see syllabus for the rest

What you will need in this course

- **A "pro" subscription to ChatGPT, Gemini, or Claude.**
- My recommendation: **Claude**
- Will it be worth it? 3x$20 = $60 (Pro). 3x$100 = $300 (Max)
- Ability to bring and work on a **laptop for codefests (Wed).**

> Regarding the question of where this leaves human grad students, my advice to students at all levels (and in any field) is to take LLMs seriously. Do not fall into the hallucination trap: "I asked the LLM X and it made something up, so I'm just going to wait for it to improve." Instead, get to know these models. Learn what they are good at and what they fail at. Buy the $20 subscription. It will change your life.

https://www.anthropic.com/research/vibe-physics

How to get help?

- **Post question on Slack**
- **E-mail:** teuscher@pdx.edu
- **Office hours:** https://www.teuscher-lab.com/officehour

- How do you like to be called?
  - Christof
  - Dr. Teuscher
  - Dr. T
  - Professor Teuscher
  - ~~Hey Teuscher!~~

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

Live the “campsite rule”

Your presence should make this classroom a better place.

- **Engage**: listen, participate, contribute, do your share. Play fair.
- **Focus**: no side conversations, no laptop and phone addiction, no work for other classes.
- **Be present**: show up, be on time, act like a professional.
- **Challenge yourself**: do always a little more instead of just the bare minimum. Up your game, set high standards for yourself. Produce work that you can be proud of.
- **Help others** to be successful. Share and care.
- **Willingness to learn**: have a growth mindset, intellectual humility and ambition.
- **Be in charge**: Put yourself in the driver’s seat, assume responsibility for your actions.


Leaving the world a better place

Add languages

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

Leaving the world a better place, often called the **campground rule**, or just **leaving things better than you found them**, is an ethical proposition that individuals should go beyond trying not to do harm in the world, and should try to remediate harms done by others.

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

Why show up?

- "*Showing up is 80 percent of life*." — Woody Allen
- It's the key to success
- The easiest way to stay on top of things.
- The easiest way to get help.
- The easiest way to learn from others.
- The easiest way to calibrate your work.
- Nothing can replace the connections you build in person.
- Please be on time. It's a matter of professionalism.

Acknowledging AI tools

- In this course, the use of AI tools does not need to be acknowledged. It is understood that you will be using them.
- However, proper credit must be given for any other source your use.
- You must be able to explain any code in your repository. The final examination is designed to verify this. LLM-generated code that you do not understand and cannot debug is a liability, not an asset.

Prep for Wednesday Codefest

1. Bring your laptop
2. Register for a “pro” ChatGPT, Gemini, or Claude subscription.
3. Set up your IDE and vibe coding workflow.
4. Accept the Slack invite.
5. Create a Github, post the link on Canvas (see spreadsheet)
6. Complete the pre-course assessment survey.